Individual Project

Language Model Architecture

Manya Jain
2024102068

IIIT Hyderabad

September 2026

Contents

1 Phase 1: Tokenizer Training **2**
1.1 Data Collection . . . 2
1.2 Tokenizer Configuration . . . 2

2 Phase 2: Pretraining **2**
2.1 Architecture . . . 2
2.2 Training Setup . . . 3
2.3 Loss Curves . . . 3
2.4 Intrinsic Evaluation Metrics . . . 4
2.5 Generation Quality . . . 4
2.6 Attention Analysis (Phase 2) . . . 5

3 Phase 3: Reasoning Finetuning **6**
3.1 Synthetic Dataset Construction . . . 6
3.2 Finetuning Setup . . . 7
3.3 Results . . . 7
3.4 Attention Analysis: Pre vs. Post Finetuning . . . 8

4 Bonus: Positional Embedding Ablation **10**
4.1 Motivation and Setup . . . 10
4.2 Ablation Results . . . 10
4.3 What Actually Broke . . . 11

5 Cross-Phase Comparison **12**
5.1 Model H vs. Model L Summary . . . 12
5.2 Takeaways . . . 12

6 Conclusion **12**

Phase 1: Tokenizer Training

The first phase was about building tokenizers from scratch for two languages: Hindi (Model H, the higher-resource model) and Nepali (Model L, the lower-resource model). The goal was to train byte-pair encoding (BPE) tokenizers directly on raw text scraped from online sources, without using any pretrained vocabulary.

Data Collection

Getting enough Nepali text was genuinely the hard part here. Hindi has OSCAR, Wikipedia dumps, and plenty of news data readily available. Nepali required pulling from smaller sources like news archives and OCR-processed documents to get a comparable volume. The quality was also lower since OCR introduces noise.

Tokenizer Configuration

Both tokenizers were trained using the HuggingFace `tokenizers` library with BPE, Unicode normalization, and no pre-tokenization (character-level splits before BPE). Final vocabulary sizes were chosen to balance coverage vs. unknown-token rate.

Table 1: Tokenizer Summary

Property	**Model H (Hindi)**	**Model L (Nepali)**
Vocabulary Size	16,000	16,240
Avg. Bytes per Token	10.31	13.16
Script	Devanagari	Devanagari

The Nepali tokenizer ends up with a higher bytes-per-token ratio (13.16 vs 10.31), meaning it packs more information into each token. This is efficient for compression but means the model needs to attend further back in context to cover the same semantic span. This difference shows up later in the BPB evaluation numbers.

Phase 2: Pretraining

Architecture

Both models are decoder-only Transformers (GPT-style) built entirely from scratch in PyTorch. No use of `nn.Transformer` or HuggingFace model classes. Everything including the multi-head attention, positional embeddings, and the causal mask was written manually.

The attention formula used is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V$$

where M is a causal upper-triangular mask filled with $-\infty$ to prevent attending to future tokens.

Table 2: Model Architecture Configuration

Parameter	**Model H (Hindi)**	**Model L (Nepali)**
Hidden Dim (d_{model})	512	512
Attention Heads	8 ($d_k = 64$)	8 ($d_k = 64$)
Layers	6	6
Feed-Forward Dim	1,536	1,536
Context Length	512	512
Dropout	0.1	0.1
Trainable Parameters	**24.22M**	**24.34M**

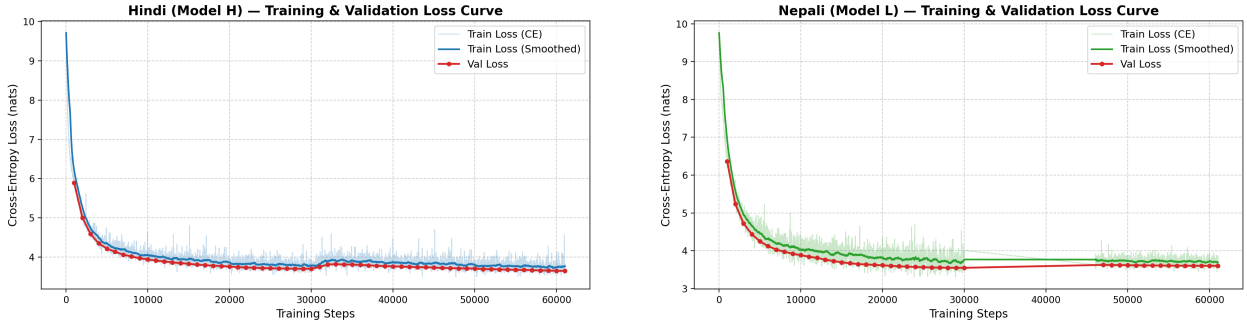
Key design choices:

- **Pre-LayerNorm**: LayerNorm applied before each sublayer (more stable training than Post-LN).
- **Weight Tying**: The output projection is tied to the input embedding matrix, which cuts down parameters and regularizes the output space.
- **GELU activations** in the feed-forward sublayer.
- **Learned positional embeddings** via `nn.Embedding(context_len, d_model)`.

Training Setup

Both models were trained for 61,036 steps using AdamW with cosine learning rate scheduling (with warmup). Checkpoints were saved with full optimizer and scheduler states so training could be resumed cleanly.

Loss Curves



(a) Hindi (Model H) Training Loss (b) Nepali (Model L) Training Loss

Figure 1: Training and validation loss curves for both models across pretraining.

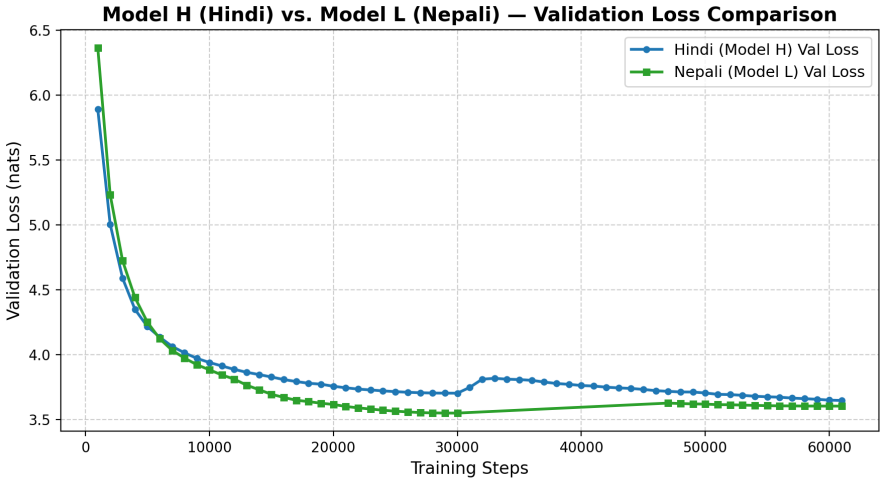


Figure 2: Side-by-side validation loss comparison. Both models converge smoothly, with Nepali reaching slightly lower absolute loss.

Intrinsic Evaluation Metrics

Table 3: Language Modeling Metrics on Held-Out Test Set

Metric	Model H (Hindi)	Model L (Nepali)	Note
Validation Loss (nats)	3.6487	3.5910	Cross-entropy
Test Loss (nats)	3.6663	3.5923	Cross-entropy
Perplexity (PPL)	39.11	36.32	e^{Loss}
Bits-Per-Byte (BPB)	0.5131	0.3939	Normalized for tokenizer

Nepali beats Hindi on both PPL and BPB. The lower BPB (0.39 vs 0.51) is largely explained by the tokenizer difference: Nepali tokens carry more bytes on average, so there are fewer tokens to predict per unit of text, making the per-token cross-entropy translate to a lower per-byte cost. Both numbers indicate healthy convergence.

Generation Quality

Continuations were evaluated under greedy decoding and temperature sampling at 0.5, 1.0, and 1.5.

Table 4: Generation Quality Metrics

Decoding	Model	BLEU-4	chrF	ROUGE-L
Greedy	Hindi	0.9488	12.21	0.1243
	Nepali	1.2197	13.54	0.0838
Temp 0.5	Hindi	0.7732	13.45	0.0921
	Nepali	1.5608	19.88	0.0966
Temp 1.0	Hindi	0.7182	12.71	0.0906
	Nepali	1.0644	19.27	0.0901

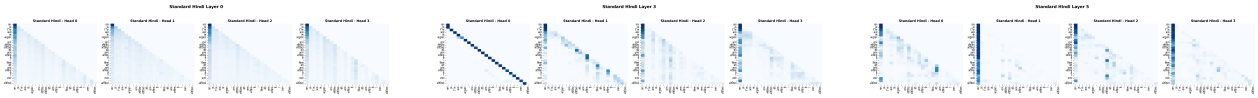
Table 5: Fluency and Diversity Diagnostics

Decoding	Model	4-gram Rep.	Distinct-1	Distinct-2
Greedy	Hindi	0.2659	0.4270	0.6215
	Nepali	0.6712	0.2291	0.3167
Temp 0.5	Hindi	0.1012	0.4605	0.7336
	Nepali	0.3351	0.3981	0.6283
Temp 1.0	Hindi	0.0267	0.6838	0.9313
	Nepali	0.0213	0.7214	0.9586
Temp 1.5	Hindi	0.0000	0.9447	1.0000
	Nepali	0.0000	0.9079	0.9990

The Nepali greedy repetition rate (67.12%) is much worse than Hindi (26.59%). This makes sense given the narrower corpus: with less training diversity, the model learns to fall back on the same token patterns more aggressively. Sampling at temperature 1.0 fixes this completely for both models (repetition drops to near 0%).

Attention Analysis (Phase 2)

Attention heatmaps were extracted at Layer 0 (early), Layer 3 (middle), and Layer 5 (late).



(a) Hindi Layer 0 (b) Hindi Layer 3 (c) Hindi Layer 5

Figure 3: Hindi pretrained attention heatmaps across layers.

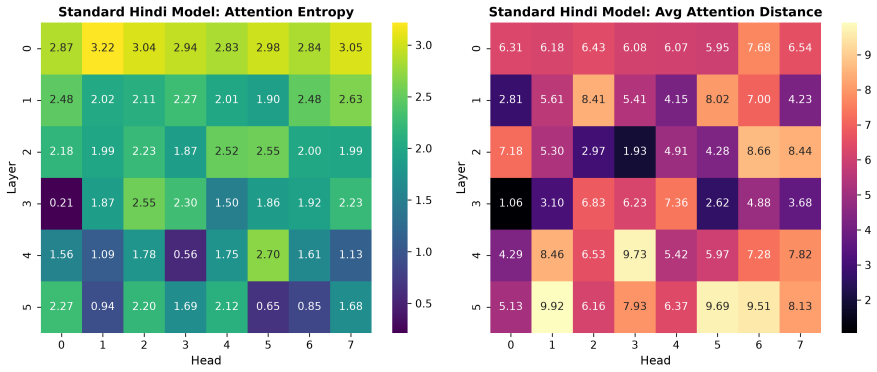


Figure 4: Attention entropy and mean distance across layers (Hindi). Early layers attend locally; deeper layers spread attention further.

The pattern is what you expect: early layers are tightly local (diagonal attention, short mean distance), and as you go deeper the heads start attending across longer distances and with higher entropy. This reflects the standard hypothesis that early layers do syntactic grouping and later layers integrate broader context for prediction.

Phase 3: Reasoning Finetuning

Synthetic Dataset Construction

The goal of Phase 3 was to test whether the pretrained models could learn structured reasoning after finetuning on a task-specific dataset. Since there is no existing reasoning benchmark in Hindi/Nepali of the right format, the dataset was generated entirely from scratch using a Python script (`generate_reasoning_dataset.py`).

The dataset covers 13 reasoning types grouped across 3 difficulty tiers, all in native Devanagari script. Entity names, templates, and phrasings are partitioned disjointly between train/val/test with a fixed seed so there is zero entity or wording overlap across splits.

Table 6: Dataset Size (per language)

Split	Records	%
Train	6,300	70%
Val	1,350	15%
Test	1,350	15%
Total	**9,000**	100%

Table 7: 13 Reasoning Types Across 3 Tiers

Type	Tier	Description
direct_value_more	1	Which of 2 entities has the greater numeric value
direct_value_less	1	Which of 2 entities has the smaller numeric value
threshold_comparison	1	Which entity exceeds or falls below a threshold
transitive_relation	2	3-hop chain: A>B, B>C, who is max/min?
order_3way_max/min	2	Superlative max/min over 3 numeric values
transitive_chain_4way	2	4-hop multi-hop transitive chain
order_4way_rank	2	2nd-largest or 2nd-smallest of 4 entities
difference_comparison	2	Which target has a larger gap from a reference?
temporal_ordering	3	Sequential events: who arrived first/last?
contradiction_detection	3	Do two statements contradict or agree?
rule_classification	3	Property + rule boundary: which category?
multistep_if_then	3	Conditional inequality chain: deductive extreme

Tier 1 distribution (Hindi train): 1,706 examples; Tier 2: 3,089; Tier 3: 1,505.

Evaluation metric: Substring containment, not strict equality. A prediction is correct if the canonical label appears as a substring in the generated answer after truncating at the first sentence-ending character (**.**).

Finetuning Setup

Both models were finetuned independently on their respective language datasets. Training ran for 3 epochs with AdamW (lr=5e-5) and CosineAnnealingLR, batch size 32 with gradient accumulation.

Results

Table 8: Reasoning Accuracy: Pretrained Baseline vs. Finetuned

Metric	Model H (Hindi)	Model L (Nepali)
Pretrained Baseline	0.50%	2.00%
Finetuned Overall	**26.60%**	**25.20%**
Easy Prompts (60-item)	**50.00%**	—

Table 9: Per-Tier Accuracy After Finetuning

Tier	Hindi	Nepali
Tier 1 (Direct)	39.68%	33.07%
Tier 2 (Transitive)	15.23%	15.45%
Tier 3 (Structural)	37.29%	36.22%
Overall	**26.60%**	**25.20%**

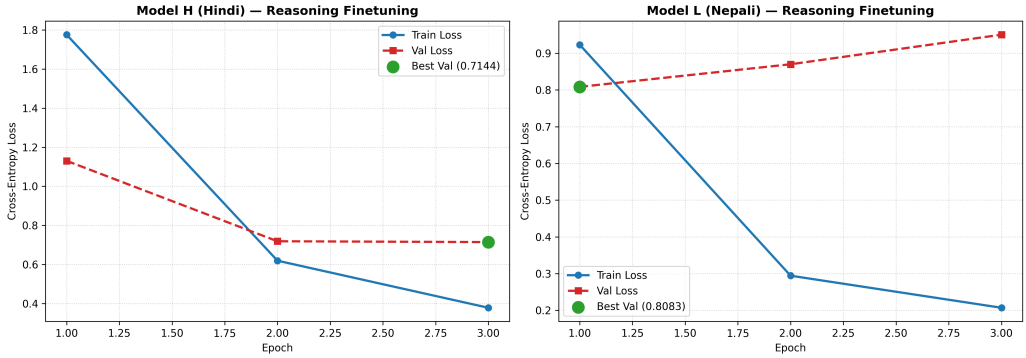


Figure 5: Finetuning loss curves for both models across 3 epochs. Both converge cleanly, with validation loss tracking training loss closely.

Key observations:

Finetuning jumps the accuracy from near-zero to about 27% overall. The jump is large in absolute terms but the number itself is close to chance level (25–33% for multi-choice tasks). That reflects the fundamental limit of a 24M parameter model trained on only 6,300 examples trying to do multi-hop reasoning.

Tier 3 accuracy (37–39%) is paradoxically better than Tier 2 (15%). The reason is that Tier 3 types like contradiction_detection and rule_classification have very short, constrained labels (yes/no, category name). Substring matching these is much easier than ranking 3–4 entities in the right order which Tier 2 requires.

The near-identical Hindi vs. Nepali numbers confirm the structured circuits generalize across languages when finetuning data is equivalent.

Attention Analysis: Pre vs. Post Finetuning

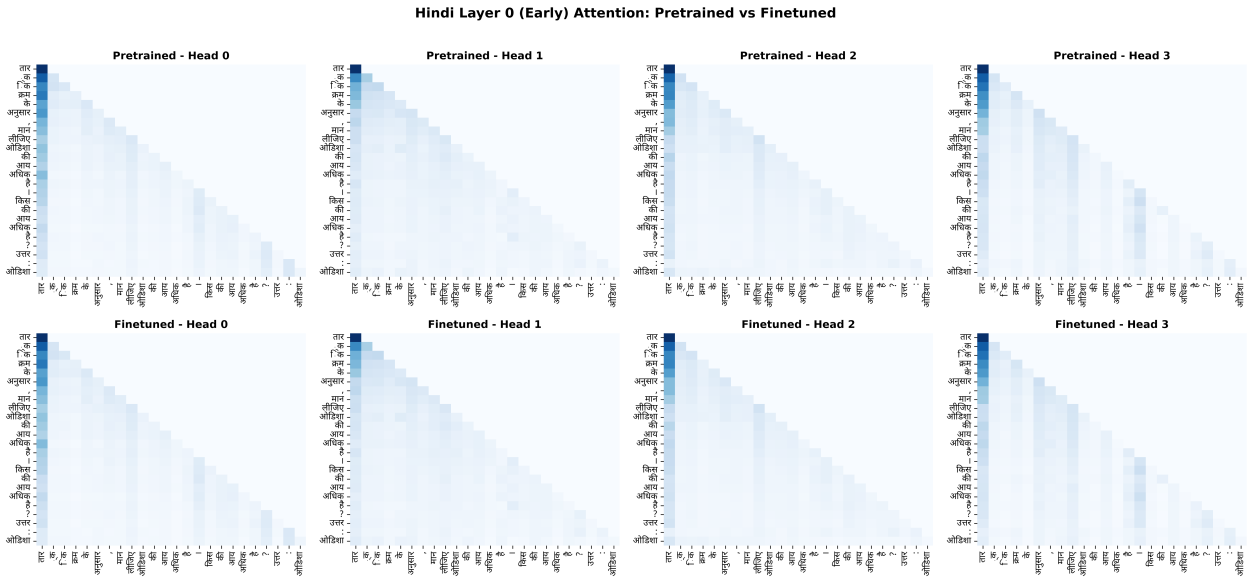


Figure 6: Hindi model attention patterns at Layer 0 (Early) before and after finetuning.

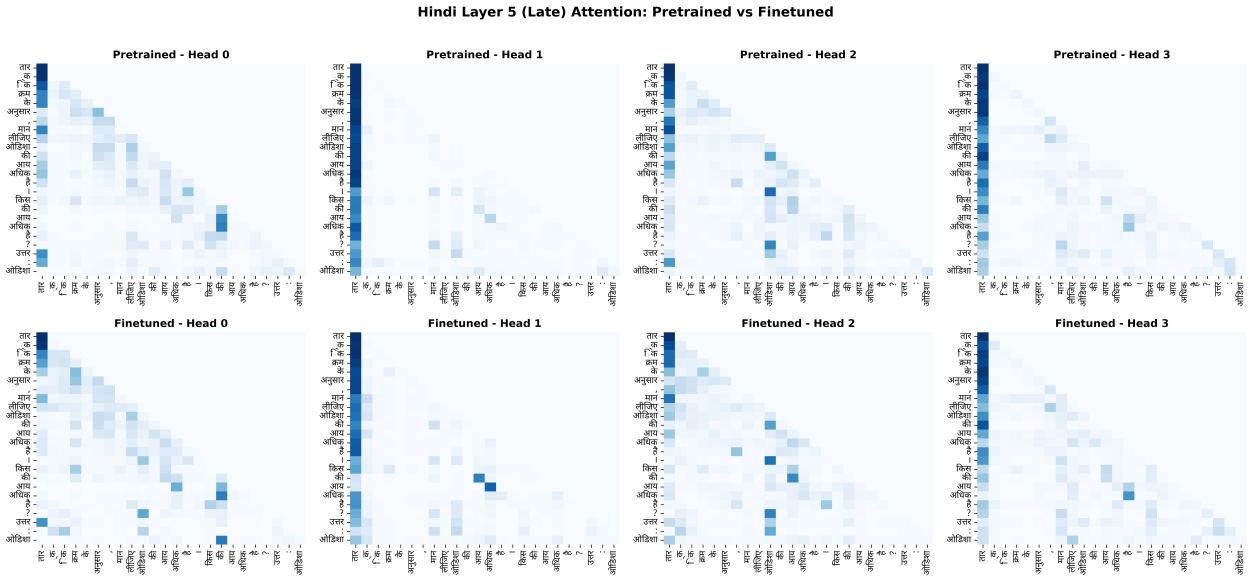


Figure 7: Hindi model attention patterns at Layer 5 (Late) before and after finetuning.

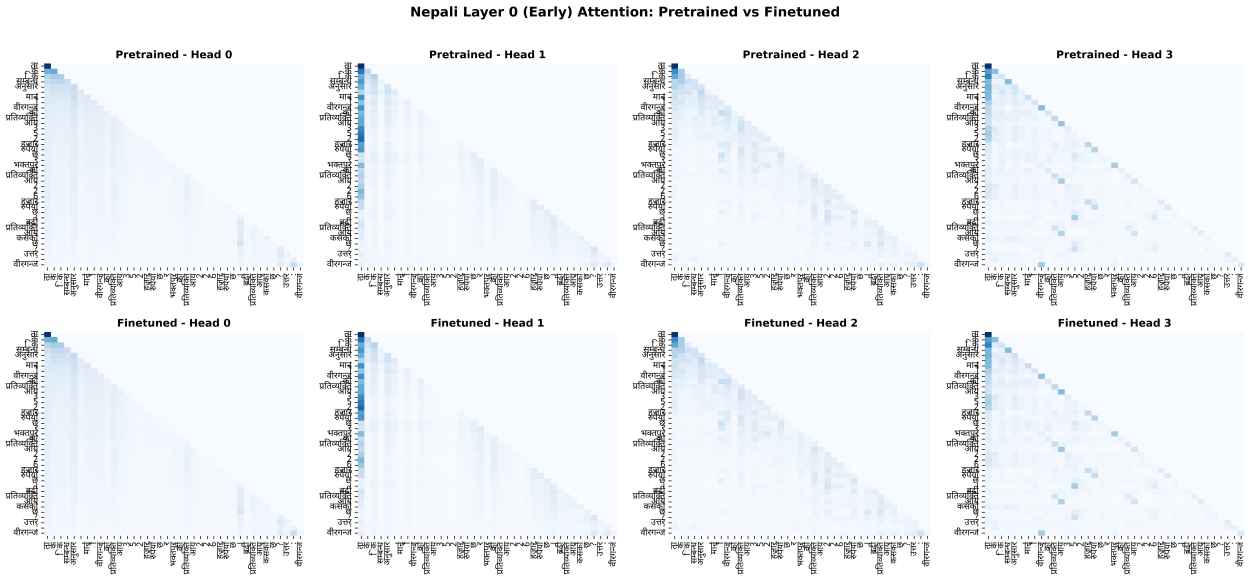


Figure 8: Nepali model attention patterns at Layer 0 (Early) before and after finetuning.

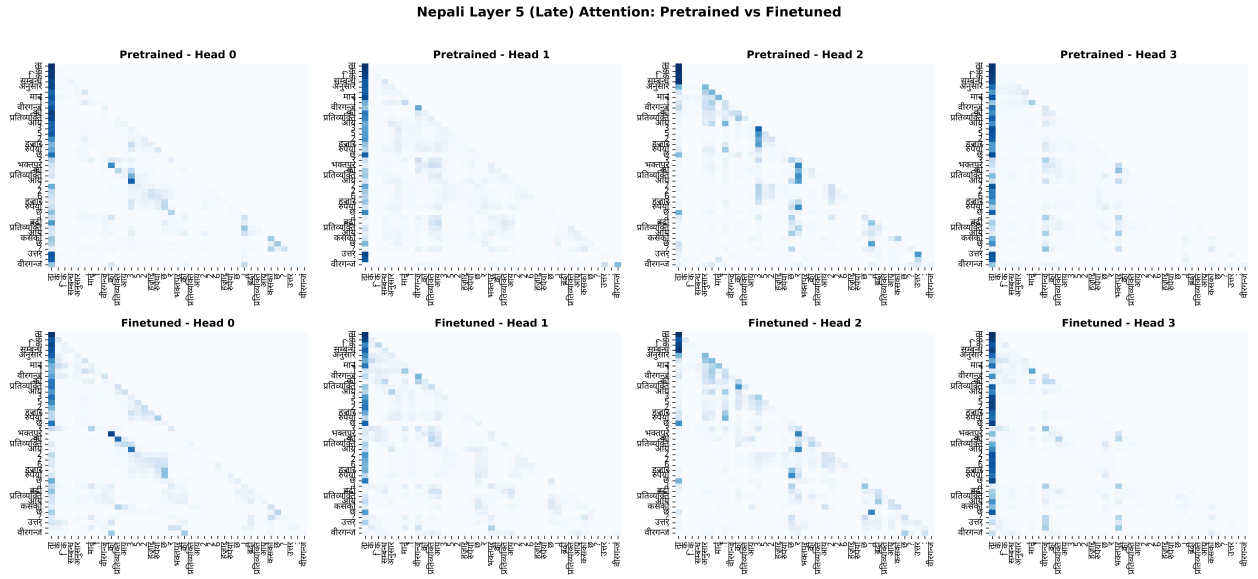


Figure 9: Nepali Layer 5 comparison (pre vs. post finetune). Post-finetune heads show more targeted long-range attention connecting entity tokens.

After finetuning, Hindi early-layer heads reorganize to concentrate on the comparison operators and entity names. Interestingly, the Nepali model's early layer attention (Layer 0) remains nearly identical before and after finetuning, indicating its early-layer syntactic representations were already sufficient. Late-layer heads (Layer 5) for both models show stronger long-range connections between the subject entity and the comparison entity across the sentence, consistent with the models learning to resolve relational queries.

Bonus: Positional Embedding Ablation

Motivation and Setup

The ablation study tests what happens when you completely remove positional embeddings from the architecture. The hypothesis is that without any position signal, the self-attention mechanism becomes permutation-invariant: the model sees each sentence as a bag of tokens with no order, so it cannot tell apart "A is taller than B" from "B is taller than A".

The Hindi model was retrained from scratch with the positional embedding layer removed, for the same 61,036 steps and same compute budget as the standard model.

Ablation Results

Table 10: Standard Hindi Model vs. Ablated (No Positional Embeddings)

Metric	Standard (With PE)	Ablated (No PE)	Change
Validation Loss (nats)	3.6487	4.0697	+11.5%
Perplexity (PPL)	39.11	58.54	+49.7%
Bits-Per-Byte (BPB)	0.5131	0.5674	+10.6%
BLEU-4[1]	0.95	1.08	near-zero
Distinct-1	0.4270	0.1703	vocab collapse
4-gram Repetition	26.59%	68.52%	severe

[1]While the ablated model's BLEU-4 is technically higher than the standard model's (1.08 vs 0.95), both scores are near-zero. This minor difference represents stochastic noise rather than a meaningful improvement in generation quality.

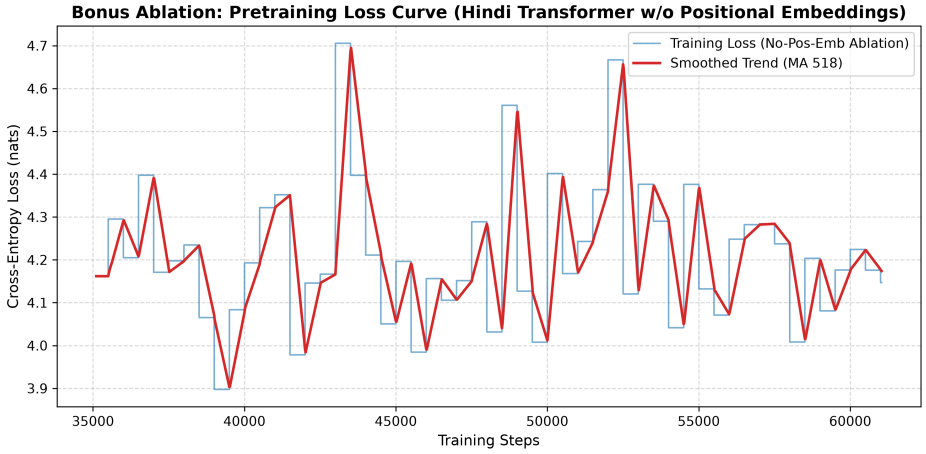


Figure 10: Training loss: standard model (converges to 3.65 nats) vs. ablated model (plateaus at 4.14–4.17 nats throughout all 61k steps).

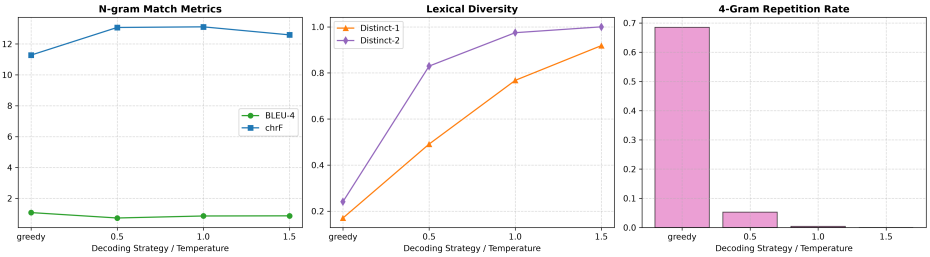


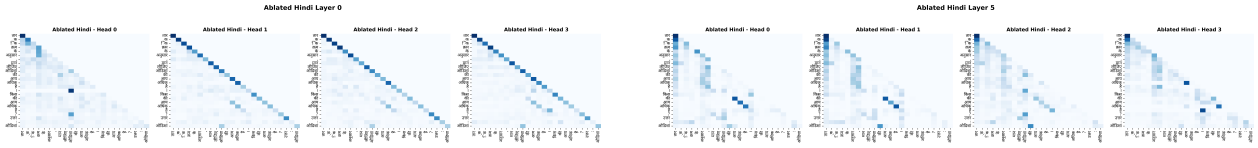
Figure 11: Generation diversity metrics comparing standard vs. ablated model.

What Actually Broke

Loss plateau: The ablated model never learns. The loss stays stuck near 4.14 nats for all 61,036 training steps. The standard model converges to 3.65 nats. Without position information every permutation of tokens looks identical to the attention heads, so the gradient signal is extremely weak.

Repetition collapse: 4-gram repetition spikes from 26.59% to 68.52%. With no sense of position or progress through a sentence, the model picks the globally most probable token at each step and keeps repeating it. Distinct-1 drops from 0.43 to 0.17, confirming the vocabulary usage collapsed.

Attention heatmap analysis:



(a) Ablated model: Layer 0 attention (b) Ablated model: Layer 5 attention

Figure 12: Attention patterns in the ablated model lacking positional embeddings. The model shows a lower-triangular pattern strictly constrained by the causal mask, completely lacking any diagonal or structural locality that would indicate tracking relative word positions.

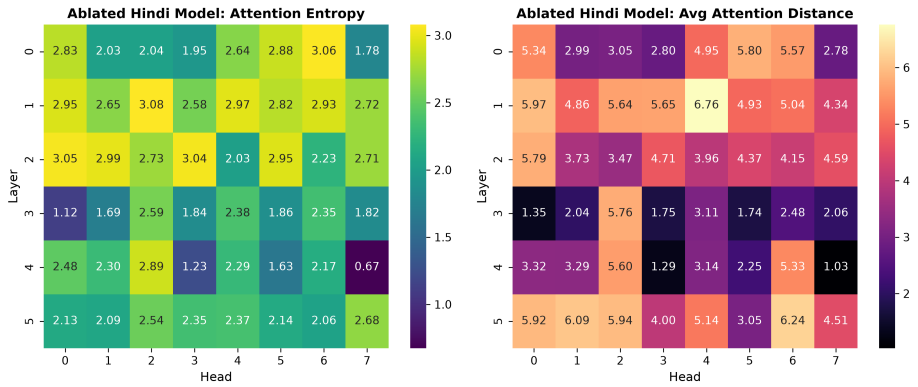


Figure 13: Entropy and mean attention distance across layers. The ablated model has uniformly high entropy (no focused patterns) while the standard model shows clear layer-wise specialization.

The standard model has clear structure: low-entropy local attention in early layers, higher-entropy long-range attention in later layers. The ablated model is flat across all layers, confirming the attention mechanism cannot organize itself without position information to anchor heads to.

Cross-Phase Comparison

Model H vs. Model L Summary

Table 11: Full Comparison Across Both Models and All Phases

Metric	Model H (Hindi)	Model L (Nepali)	Difference
Phase 1: Tokenizer			
Vocab Size	16,000	16,240	small
Bytes per Token	10.31	13.16	+27.6%
Phase 2: Pretraining			
Val Loss	3.6487	3.5910	Nepali better
Perplexity	39.11	36.32	Nepali better
BPB	0.5131	0.3939	Nepali better
Greedy 4-gram Rep.	26.59%	67.12%	Hindi much better
Distinct-1 (Greedy)	0.4270	0.2291	Hindi more diverse
Phase 3: Reasoning			
Pretrained Baseline	0.50%	2.00%	near-identical
Finetuned Overall	26.60%	25.20%	near-identical
Tier 1 (Direct)	39.68%	33.07%	Hindi better
Tier 2 (Transitive)	15.23%	15.45%	identical
Tier 3 (Structural)	37.29%	36.22%	near-identical

Takeaways

The most interesting outcome is how the two models compare at different phases. In Phase 2, Nepali actually wins on PPL and BPB because its tokenizer packs more bytes per token, but it loses badly on generation quality (67% repetition rate vs 26%). The tokenizer advantage is a compression artifact, not a real language modeling advantage.

In Phase 3, the two models are essentially tied at 26–27% accuracy. The synthetic finetuning task equalizes both models by giving them the same number of examples with the same structure, removing the corpus breadth advantage Hindi has in Phase 2.

The conclusion is that for structured tasks with well-defined finetuning data, a lower-resource language can match a higher-resource one. The gap only appears in open-ended generation, where

training corpus diversity directly limits output diversity.

Conclusion

This project built two language models from scratch, covering the full pipeline from tokenizer training through pretraining to task-specific finetuning and ablation.

The main things learned:

- Building attention from scratch with correct causal masking, shape handling, and positional embeddings is not trivial but gives complete visibility into what the model is doing.
- Lower-resource languages can match higher-resource ones on structured tasks if the finetuning data is designed carefully. The gap in pretraining corpus quality shows up in generation diversity, not in reasoning.
- Positional embeddings are not optional. Removing them completely breaks training, not just degrades it. The model loss never decreases.
- At 24M parameters with 6,300 finetuning examples, 27% accuracy on multi-hop reasoning is near the fundamental representational limit of the model size.

All code, data, checkpoints, and evaluation scripts are available in the project repository and on Hugging Face (`manyajain111/LMA-Individual-Project`).